

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Разработка драйвера блочного устройства с автоматической коррекцией ошибок

Смирнов Алексей Артёмович

Научный руководитель: Ловягин Никита Юрьевич

23 июня 2026 г.

Цель работы

Проблема повреждения данных остаётся актуальной даже для современных накопителей. Причины возникновения ошибок:

- деградация носителя;
- ошибки контроллера;
- сбои питания;
- ошибки передачи данных;
- silent data corruption.

Цель работы:

Разработка драйвера блочного устройства Linux, обеспечивающего контроль целостности данных на уровне блочного ввода-вывода.

Постановка задачи

Требуется разработать драйвер блочного устройства для Linux, который:

- функционирует как промежуточный слой между файловой системой и накопителем;
- прозрачен для пользовательских приложений;
- вычисляет и хранит контрольные суммы данных;
- проверяет целостность данных при чтении;
- обнаруживает повреждения данных и сообщает об ошибке;
- обеспечивает корректную работу при параллельном доступе.

Перспектива развития

Архитектура должна позволять последующее внедрение кодов исправления ошибок (ECC).

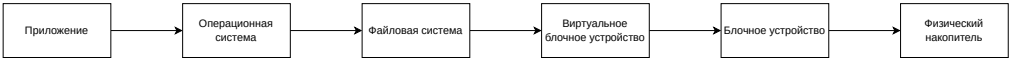
Существующие решения

Решение	Контроль целостности	Независимость от ФС	Самостоятельность
RAID	Частично	Да	Да
Btrfs/ZFS	Да	Нет	Частично
dm-integrity	Да	Да	Нет
Разрабатываемый драйвер	Да	Да	Да

Основные требования:

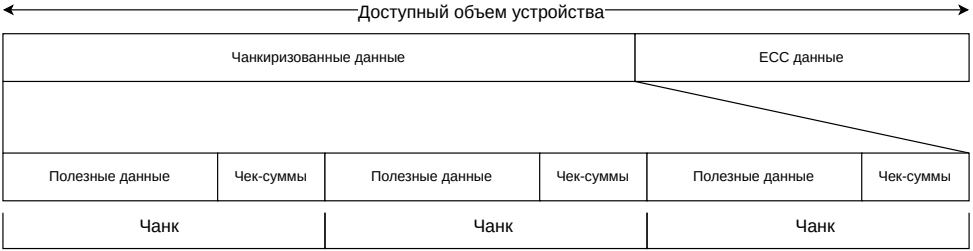
- прозрачность для приложений;
- независимость от файловой системы;
- самостоятельное выявление и исправление ошибок.

Место драйвера в потоке обработки ввода–вывода



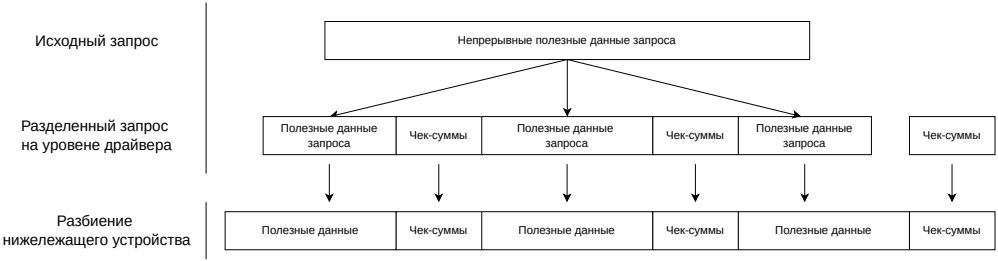
- драйвер реализован как Device Mapper target;
- создаёт виртуальное блочное устройство поверх требуемого;
- получает операции чтения и записи, обрабатывает и отправляет дальше.

Размещение данных и метаданных



В дальнейшем развитии проекта будет добавлена возможность статически определять соотношение (полезных данных)/(контрольных сумм)/(ECC метаданных) и проведены измерения влияния выбранных соотношений на характеристики драйвера.

Следствие архитектуры размещения данных и метаданных



Представление разбиения верхнего запроса на запросы к устройству

Синхронизация доступа

- единица синхронизации — чанк;
- используется `rw_semaphore`;
- блокировки создаются лениво;
- структура блокировок хранится в `xarray`.

Преимущества:

- поддержка параллельного чтения;
- отсутствие больших затрат памяти;
- масштабируемость на устройства большого объёма.

Обработка операции записи

Основные этапы:

1. разбиение запроса по чанкам;
2. инициализация структур запросов;
3. вычисление контрольных сумм;
4. формирование запросов к устройству;
5. эксклюзивная блокировка затронутых чанков;
6. запись данных и метаданных;
7. разблокировка затронутых чанков;
8. завершение запроса.

Обработка операции чтения

Основные этапы:

1. разбиение запроса по чанкам;
2. инициализация структур запросов;
3. формирование запросов к устройству;
4. совместная блокировка затронутых чанков;
5. чтение данных и метаданных;
6. разблокировка затронутых чанков;
7. вычисление и проверка контрольных сумм;
8. завершение запроса.

Проверка корректности

Проведены следующие тесты:

- работа с файловой системой ext4;
- искусственное повреждение метаданных;
- тестирование конкурентного доступа.

Результаты:

- корректная работа чтения и записи;
- успешная работа с ext4;
- обнаружение повреждения данных.

Все End-to-End тесты показали корректность выполнения функциональных требований.

Аппаратно–программная среда выполнения

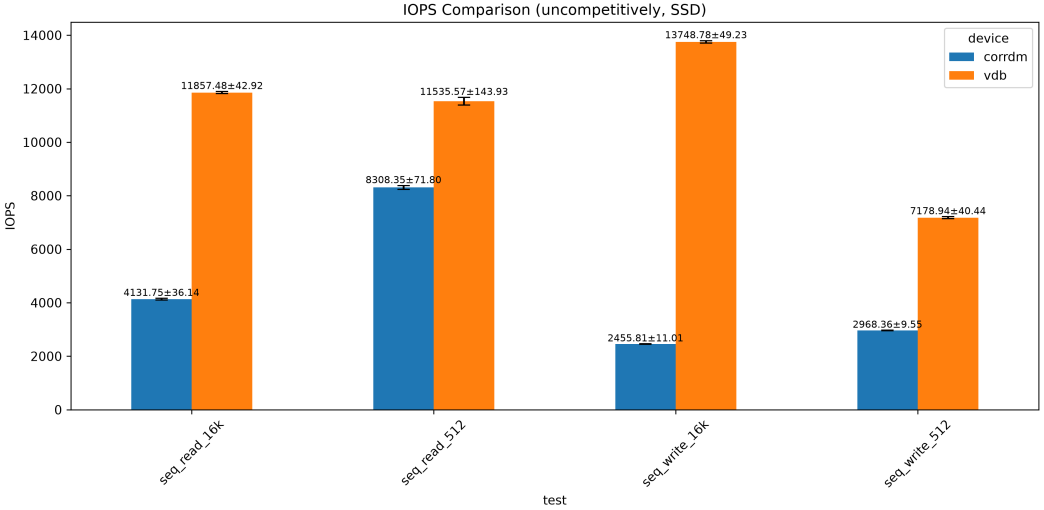
Аппаратная среда выполнения:

- Процессор — Intel(R) Core(TM) i7-8700K (12) @ 4.70 GHz;
- Оперативная память — 2x Corsair Vengeance LPX DDR4 16Gb;
- Накопитель — Samsung SSD 980 1TB;
- Интерфейс подключения — PCI Express 3.0 x4.

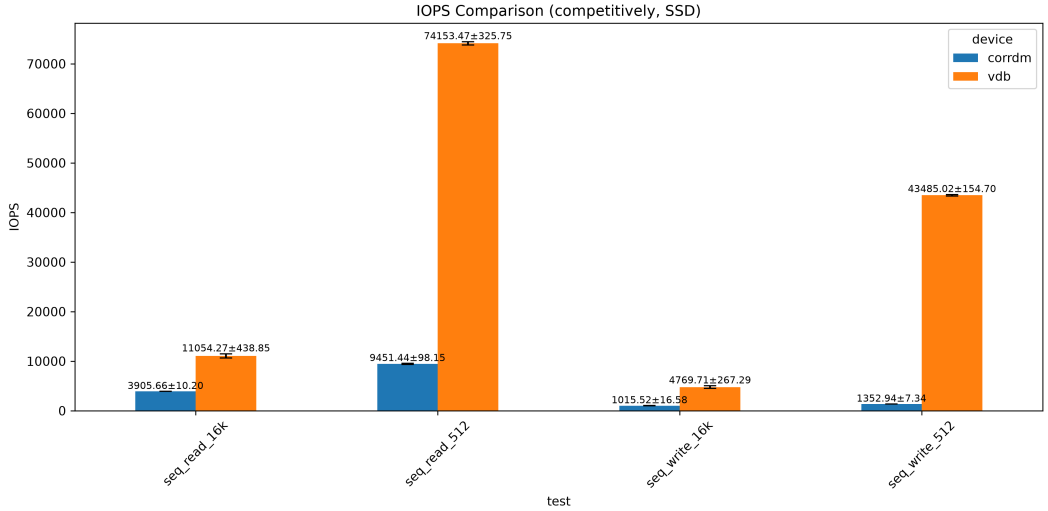
Программная среда выполнения (QEMU/KVM):

- Операционная система — Fedora Linux 42 (Server Edition) x86_64;
- Ядро Linux — 6.19.11-100.fc42.x86_64;
- Версия fio — 3.37;

Производительность



Производительность



Профилирование функций

Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
dm_map	72390	853847.450	11.795
trn_rq_init	72390	634219.215	8.761
complete_trn_rq	72390	237042.380	3.275
submit_work	289738	878795.210	3.033
trn_rq_submit	72390	204359.009	2.823
complete_work	289738	614196.910	2.120
trn_p_rq_init	289738	552647.246	1.907
trn_mw_rq_init	289200	379183.103	1.311
trn_p_rq_end_io	579476	392736.040	0.678
bio_crc_calc.isra.0	289200	147325.347	0.509
complete_trn_mw_rq	289200	145350.434	0.503
locker_put_lock	289738	116133.801	0.401
locker_get_lock	289738	104439.177	0.360
metadata_bio_init	289738	38924.858	0.134

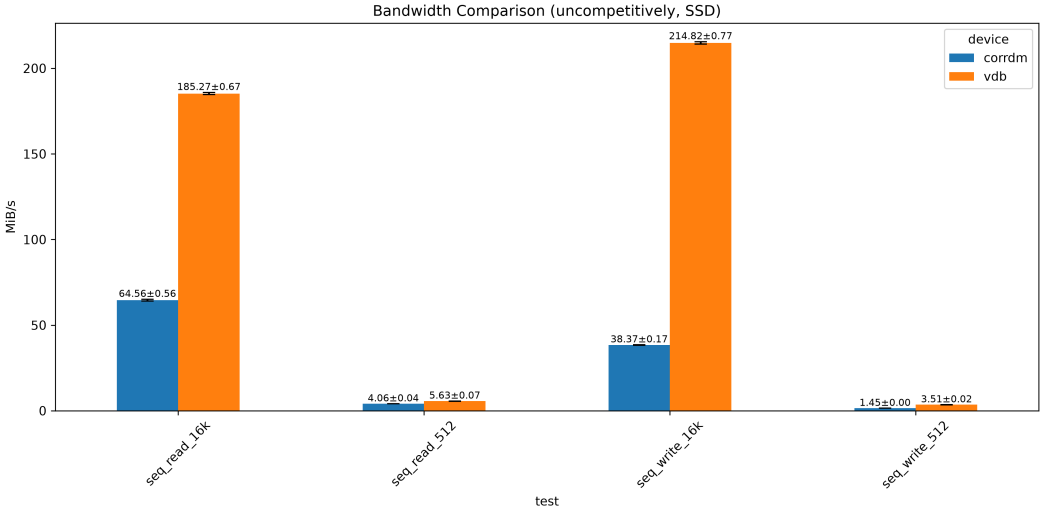
Профилирование функций и время выполнения для однопоточного теста записи блоками 16 KiB.

Профилирование функций

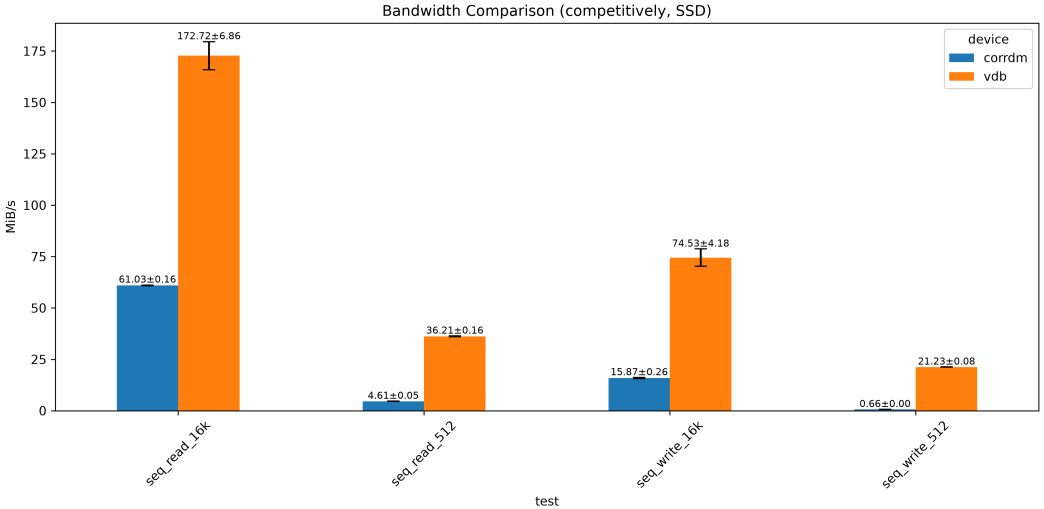
Функция	**Количество вызовов**	**Общее время (μs)**	**Среднее время (μs)**
submit_work	996960	27993735963.000	28079.096
dm_map	249151	4812636.900	19.316
trn_rq_init	249151	2996694.400	12.028
trn_rq_submit	249151	1744903.500	7.003
complete_trn_rq	249151	1161011.900	4.660
complete_work	996960	3037188.600	3.046
trn_p_rq_init	996960	2589077.400	2.597
trn_mw_rq_init	995884	1796320.800	1.804
trn_p_rq_end_io	1993920	2723595.600	1.366
bio_crc_calc.isra.0	995884	795797.250	0.799
complete_trn_mw_rq	995884	618743.270	0.621
locker_put_lock	996960	551439.210	0.553
locker_get_lock	996960	449331.050	0.451
metadata_bio_init	996960	209956.730	0.211

Профилирование функций и время выполнения для многопоточного теста записи блоками 16 KiB.

Пропускная способность



Пропускная способность



Проблемы и направления улучшения архитектуры

Основные ограничения текущей реализации:

- длительное удержание блокировок чанков;
- большое количество дополнительных операций ввода-вывода на нижележащее устройство.

Выявленные направления улучшения архитектуры:

- внедрение внутреннего кэша драйвера;
- объединение операций ввода-вывода, добавление очередей и планировщика;
- уменьшение времени удержания блокировок за счет кэша.

Дальнейшее развитие проекта

Полученные результаты:

- реализован драйвер Device Mapper;
- обеспечен контроль целостности данных;
- реализована синхронизация конкурентного доступа;
- подтверждена работоспособность на ext4.

Дальнейшее развитие:

- оптимизация текущего решения;
- внедрение восстановления данных за счет ECC;
- добавление конфигурирования драйвера;
- расширение области применимости.

Спасибо за внимание!

